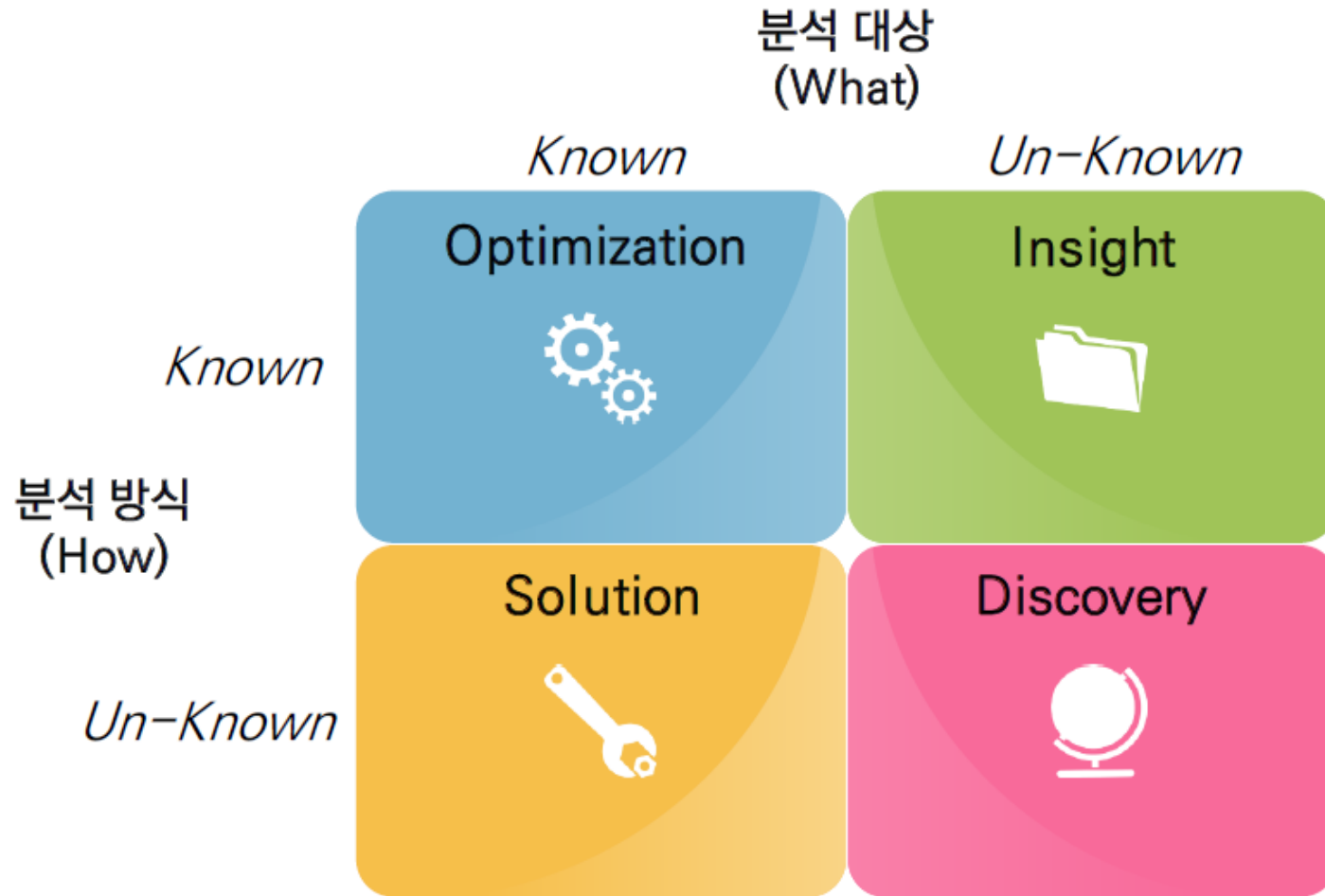


머신 러닝

(Machine Learning)

데이터 분석에서 가장 중요한 것은?



Insight vs Optimization

Insight



탐색적 자료 분석(Exploratory Data Analysis)

데이터의 특징과 내재하는 구조적인 관계를 알아내기 위한 분석 기법으로 이러한 자료의 탐색 과정을 통하여 얻은 정보를 기초로 통계모형을 세울 수 있음
미지의 특성을 파악하고 자료구조를 파악할 수 있는 증거 수집의 과정

Looking at data to see what it seems to say. It's concentrates on simple arithmetic and easy-to-draw picture. *John Tukey, 1977*



Optimization

기계학습(Machine Learning)

어떻게 하면 더 빨리 학습시키고 어떻게 하면 더 정확히 예측할 수 있을까에 대한 연구를 하며 데이터 전처리, 파생 변수 추가, 모형 선택 등의 방법을 통해서 예측 모형의 평가 점수를 높이는 과정

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E . *Tom. Mitchell, 1997*

분석을 잘 하려면?



데이터 탐색 및 시각화

의미 있는 정보는 무엇이 있을까?

- 데이터를 잘 다룰 수 있어야 함
- 데이터의 부분집합 추출 및 병합 등의 작업이 주를 이룸
- 데이터 시각화를 통해 탐색 결과를 이해하기 쉽도록 함

파이썬의 패키지

- ❖ Numpy & Pandas : 데이터를 다루기 위한 패키지
- ❖ Matplotlib & Seaborn : 데이터를 시각화 하기 위한 패키지

예측력이 높은 모형 생성

어떻게 하면 모형이 예측을 잘 할 수 있을까?

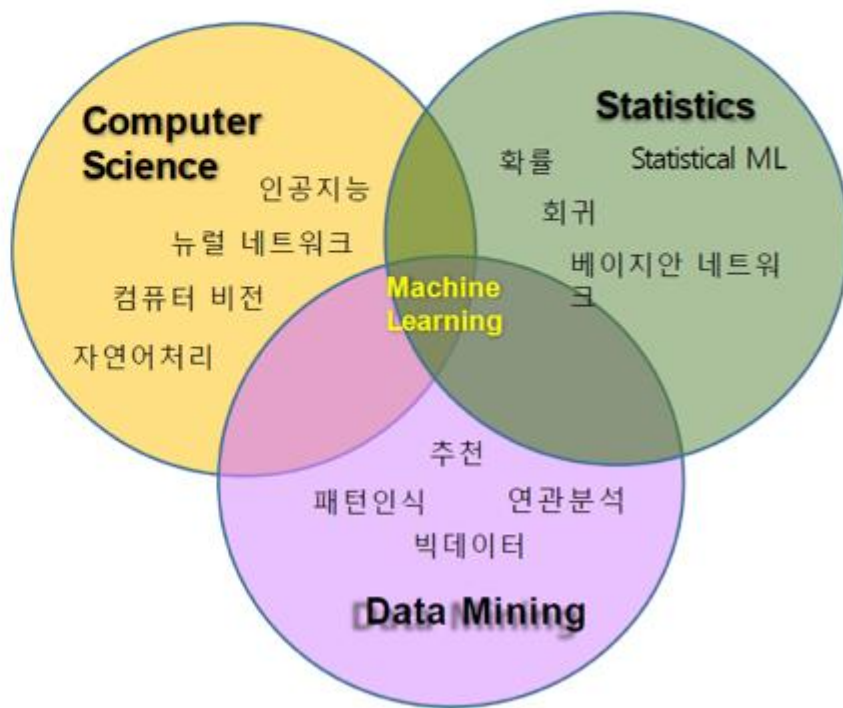
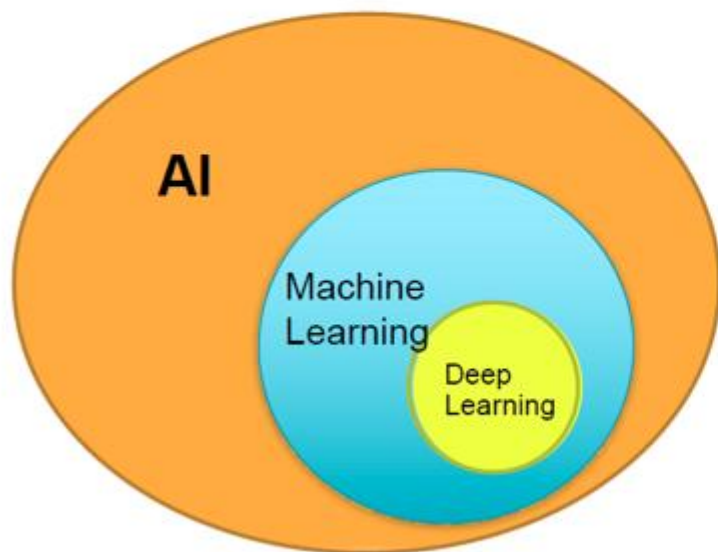
- 데이터 전처리, 파생변수 추가
- 머신러닝 모형 생성 및 예측
- 모형 평가

파이썬의 패키지

- ❖ Scikit-learn : 기계학습 라이브러리
- ❖ Statsmodels : 통계 라이브러리

머신러닝의 개념

인공지능 > 머신러닝 > 딥러닝



머신러닝의 개념

- 어플리케이션을 수정하지 않고도, 데이터를 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법을 통칭
- 데이터를 기반으로 통계적인 신뢰도를 강화하고, 예측 오류를 최소화하기 위한 다양한 수학적 기법을 적용해 데이터 내의 패턴을 스스로 인지하고, 신뢰도 있는 예측 결과를 도출
- 데이터 분석 영역은 재빠르게 머신러닝 기반의 예측 분석(Predictive Analysis)으로 재편되고 있음
- 많은 데이터 분석가와 데이터 과학자가 머신러닝 알고리즘 기반의 새로운 예측 모델을 이용해 더욱 정확한 예측 및 의사 결정을 도출하고 있으며, 데이터에 감춰진 새로운 의미와 통찰력(insight)를 발굴해 놀랄 만한 이익으로 연결시키고 있음
- 귀납적 학습 : 데이터만 주어진더라도 구조를 추론하려고 시도하기 때문

머신러닝의 분류

- 지도학습(Supervised Learning)

- 회귀(Regression) : 단순 선형 회귀 / 다항 회귀
- 분류(Classification) : 이진 분류(로지스틱 회귀) / 다중 분류

- 비지도학습(Un-supervised Learning)

- 군집(cluster) 분석 : ex) k 평균 알고리즘
- 차원 축소 : ex) 주성분 분석

머신러닝 모델 개선

- 최적의 머신러닝 알고리즘과 모델 파라미터를 구축하는 능력도 중요하지만,
- 데이터를 이해하고 효율적으로 가공, 처리, 추출해서 최적의 데이터를 기반으로 알고리즘을 구동할 수 있도록 준비하는 능력이 무엇보다 더 중요
- 다양하고 광대한 데이터를 기반으로 만들어진 머신러닝 모델은 더 좋은 품질을 약속
- 앞으로 많은 회사의 경쟁력은 어떠한 품질의 데이터로 만든 머신러닝 모델이냐에 따라 결정

머신러닝 모델의 성능 평가 - 혼동행렬(Confusion Matrix)

- 혼동행렬이란 특정 분류 모델의 성능을 평가하는 지표로, 실제값과 모델이 예측한 예측값을 한 눈에 알아볼 수 있게 배열한 행렬

		예측 클래스	
		Positive	Negative
실제 클래스	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

- 혼동행렬의 구성 요소
 - True Positive(TP): 실제 값이 참이고 모델의 예측 값도 참인 경우
 - True Negative(TN): 실제 값도 거짓이고 모델의 예측 값도 거짓인 경우
 - False Negative(FN): 실제 값은 참이나 모델의 예측 값이 거짓인 경우
 - False Positive (FP): 실제 값은 거짓이나 모델의 예측 값이 참인 경우

머신러닝 모델의 성능 평가 - 분류 모델 평가 지표

- 혼동행렬에 기반한 분류 모델 평가 지표

1. 정확도(Accuracy): 모델이 전체 중에서 예측을 올바르게 한 비율

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FN} + \text{FP} + \text{TN}}$$

2. 정밀도(Precision): 모델의 예측값이 얼마나 정확하게 예측됐는가를 나타내는 지표. 정밀도가 낮을 경우 코로나가 아닌 사람에게 코로나라고 했으니 불필요한 치료가 발생할 수 있음.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

3. 재현율(Recall): 실제 참인 값 중 모델이 참으로 예측한 값의 비율. 실제 코로나 환자들이 진단 키드 검사 시 코로나라고 예측될 확률이고, 코로나 환자를 조기에 정확하게 발견해서 신속하게 처방함이 목적이고, 재현율이 낮을 경우, 코로나인 사람에게 코로나가 아니라고 하였으니 심각한 결과를 초래할 것.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

4. F1 점수(F1-Score): 정밀도와 재현율의 조화 평균

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

머신러닝 모델의 성능 평가 - 혼동행렬 예

A 회사 코로나 진단 키트

	양성	음성
양성	9	1
음성	30	60

B 회사 코로나 진단 키트

	양성	음성
양성	1	9
음성	20	70

- A 회사 코로나 진단 키트의 정밀도 : $9 / 39 = 23\%$
 - B 회사 코로나 진단 키트의 정밀도 : $1 / 21 = 4.7\%$
- ※ 정밀도를 기준으로 더 나은 진단 키트 : A

- A 회사 코로나 진단 키트의 재현율 : $9 / 10 = 90\%$
 - B 회사 코로나 진단 키트의 재현율 : $1 / 10 = 10\%$
- ※ 재현율을 기준으로 더 나은 진단 키트병원 : A

sklearn의 train_test_split() 사용법

● 개요

- 머신러닝 모델을 학습하고 그 결과를 검증하기 위해서는 원래의 데이터를 Training, Validation, Testing의 용도로 나누어 다뤄야 함
- Training에 사용한 데이터를 검증용으로 사용하면 시험문제를 알고 있는 상태에서 공부를 하고, 그 지식을 바탕으로 시험을 치르는 꼴이므로 제대로 된 검증이 이루어지지 않기 때문
- 딥러닝을 제외하고도 다양한 기계 학습과 데이터 분석 툴을 제공하는 scikit-learn 패키지 중 model_selection에는 데이터 분할을 위한 train_test_split 함수가 들어 있음

sklearn의 train_test_split() 사용법

● Parameter & Return

```
from sklearn.model_selection import train_test_split  
  
train_test_split(arrays, test_size, train_size, random_state, shuffle, stratify)
```

(1) Parameter

- **arrays** : 분할시킬 데이터를 입력 (*Python list, Numpy array, Pandas dataframe 등..*)
- **test_size** : 테스트 데이터셋의 비율(float)이나 갯수(int) (*default = 0.25*)
- **train_size** : 학습 데이터셋의 비율(float)이나 갯수(int) (*default = test_size의 나머지*)
- **random_state** : 데이터 분할시 셔플이 이루어지는데 이를 위한 시드값 (*int나 RandomState로 입력*)
- **shuffle** : 셔플 여부 설정 (*default = True*)
- **stratify** : 지정한 data의 비율을 유지한다. 예를 들어, Label Set인 Y가 25%의 0과 75%의 1로 이루어진 Binary Set일 때, stratify=Y로 설정하면 나누어진 데이터셋들도 0과 1을 각각 25%, 75%로 유지한 채 분할된다.

(2) Return

X_train, X_test, Y_train, Y_test : arrays에 데이터와 레이블을 둘 다 넣었을 경우의 반환이며, 데이터와 레이블의 순서쌍은 유지

X_train, X_test : arrays에 레이블 없이 데이터만 넣었을 경우의 반환

sklearn의 train_test_split() 사용법

- Example

```
import numpy as np from sklearn.model_selection
import train_test_split
```

```
X = [[0,1],[2,3],[4,5],[6,7],[8,9]]
```

```
Y = [0,1,2,3,4]
```

```
# 데이터(X)만 넣었을 경우
```

```
X_train, X_test = train_test_split(X, test_size=0.2, random_state=123)
```

```
# X_train : [[0,1],[6,7],[8,9],[2,3]]
```

```
# X_test : [[4,5]]
```

```
# 데이터(X)와 레이블(Y)을 넣었을 경우
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.33, random_state=321)
```

```
# X_train : [[4,5],[0,1],[6,7]]
```

```
# Y_train : [2,0,3]
```

```
# X_test : [[2,3],[8,9]]
```

```
# Y_test : [1,4]
```

머신러닝 알고리즘

알고리즘 선정 이유

- 머신러닝은 수집된 데이터와 특정 알고리즘을 기반으로 최적의 추론을 구하는 컴퓨터 프로그램
- 최적의 추론을 구하기 위해 다양한 알고리즘들이 사용될 수 있음
- ‘어떤 알고리즘을 선정할까?’라는 질문에, 수집된 데이터의 특징과 상황에 따라 가장 적합한 알고리즘을 선택하는 것이 중요

대표적 머신러닝 알고리즘

- 선형 회귀(Linear Regression)
- 로지스틱 회귀(Logistic Regression)
- k-최근접 이웃(k-Nearest Neighbor, kNN)
- 서포트 벡터머신(Support Vector Machine, SVM)
- 의사결정 트리(Decision Tree)
- 나이브 베이즈(Naïve Bayes)
- 앙상블(Ensemble)
- 군집화(Clustering)
- 주성분 분석(Principal Component Analysis, PCA)
- 딥러닝(Deep Neural Network, DNN)